

CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA – PAULA SOUZA
ESCOLA TÉCNICA ESTADUAL FERNANDO PRESTES
DESENVOLVIMENTO DE SISTEMAS

Beatriz de Andrade Leite
Edgar Guimarães de Carvalho
Matheus Machado Sprengel
Sophia Rosa da Silva Machado
Théo Vinícius Garcia Tonche

E-commerce Full Stack

SOROCABA – SP

2026

Sumário

1 INTRODUÇÃO	1
2 OBJETIVO.....	2
3 DESENVOLVIMENTO	2
4 MODELAGEM DO SISTEMA	3
5 CONSIDERAÇÕES FINAIS.....	13
REFERÊNCIAS	14

1 INTRODUÇÃO

O comércio eletrônico tem se consolidado como uma das principais formas de compra e venda de produtos e serviços na atualidade. Com o avanço das tecnologias web e o crescimento do acesso à internet, empresas de diversos segmentos passaram a investir em plataformas digitais para ampliar seu alcance e oferecer maior comodidade aos consumidores.

Neste contexto, o presente projeto consiste no desenvolvimento de uma aplicação de e-commerce Full Stack, integrando tecnologias modernas para a construção de uma plataforma completa de vendas online. O sistema foi desenvolvido utilizando React no Frontend, Node.js e Express.js no Backend, além do banco de dados MongoDB para armazenamento das informações.

A aplicação permite que usuários realizem cadastro, autenticação, consulta de produtos, gerenciamento de carrinho de compras e acompanhamento de pedidos. Além disso, disponibiliza funcionalidades administrativas para o gerenciamento de produtos e pedidos, proporcionando uma experiência próxima à encontrada em sistemas de comércio eletrônico reais.

O desenvolvimento deste projeto possibilitou a aplicação prática de conceitos relacionados à programação web, desenvolvimento de APIs REST, autenticação de usuários, manipulação de banco de dados e integração entre diferentes camadas de software.

2 OBJETIVO

2.1 Objetivo Geral

Desenvolver uma aplicação web de comércio eletrônico utilizando arquitetura Full Stack, permitindo o gerenciamento de produtos, usuários e pedidos por meio de uma interface moderna e integrada a uma API REST.

2.2 Objetivos Específicos

- Desenvolver uma interface responsiva para navegação dos usuários;
- Implementar funcionalidades de cadastro e autenticação de usuários;
- Permitir a visualização e pesquisa de produtos disponíveis;
- Implementar um sistema de carrinho de compras;
- Desenvolver funcionalidades para criação e gerenciamento de pedidos;
- Criar uma área administrativa para gerenciamento de produtos;
- Integrar o sistema a um banco de dados MongoDB;
- Aplicar conceitos de segurança utilizando autenticação baseada em JWT;
- Utilizar boas práticas de desenvolvimento Full Stack.

3 DESENVOLVIMENTO

O projeto foi desenvolvido seguindo a arquitetura cliente-servidor, separando as responsabilidades entre Frontend, Backend e Banco de Dados

O Frontend foi construído utilizando React, responsável pela interface gráfica e interação com o usuário. A aplicação permite que os clientes naveguem pelos produtos, realizem pesquisas, adicionem itens ao carrinho e efetuem pedidos.

O Backend foi desenvolvido utilizando Node.js e Express.js, responsáveis pelo processamento das regras de negócio e pela disponibilização dos serviços por meio de uma API REST. Essa camada realiza o controle das operações de autenticação, gerenciamento de produtos, usuários e pedidos.

Para o armazenamento das informações foi utilizado o MongoDB, banco de dados não relacional amplamente utilizado em aplicações web modernas. A

comunicação entre a aplicação e o banco é realizada através da biblioteca Mongoose, que facilita a modelagem e manipulação dos dados.

A autenticação dos usuários foi implementada utilizando JSON Web Token (JWT), garantindo maior segurança no acesso às funcionalidades protegidas do sistema. As senhas dos usuários são armazenadas de forma criptografada utilizando a biblioteca BCrypt.

Durante o desenvolvimento foram aplicados conceitos de componentização, roteamento, gerenciamento de estado e integração entre Frontend e Backend. Além disso, foram elaborados diagramas de casos de uso, classes, sequência e arquitetura para representar visualmente a estrutura e o funcionamento do sistema.

4 MODELAGEM DO SISTEMA

4.1 Diagrama de Arquitetura

O sistema foi desenvolvido utilizando uma arquitetura cliente-servidor, composta por três camadas principais: Frontend, Backend e Banco de Dados. Essa arquitetura permite a separação das responsabilidades do sistema, facilitando a manutenção, escalabilidade e organização do código.

O Frontend foi desenvolvido utilizando React, sendo responsável pela interface gráfica e interação com o usuário. O Backend foi implementado com Node.js e Express.js, fornecendo uma API REST para processamento das regras de negócio e comunicação com o banco de dados MongoDB.

As requisições realizadas pelos usuários são enviadas pelo Frontend para a API, que processa as informações por meio dos controladores do sistema, responsáveis pelo gerenciamento de usuários, produtos e pedidos. Os controladores utilizam os modelos de dados para realizar operações de armazenamento, consulta e atualização das informações no banco de dados.

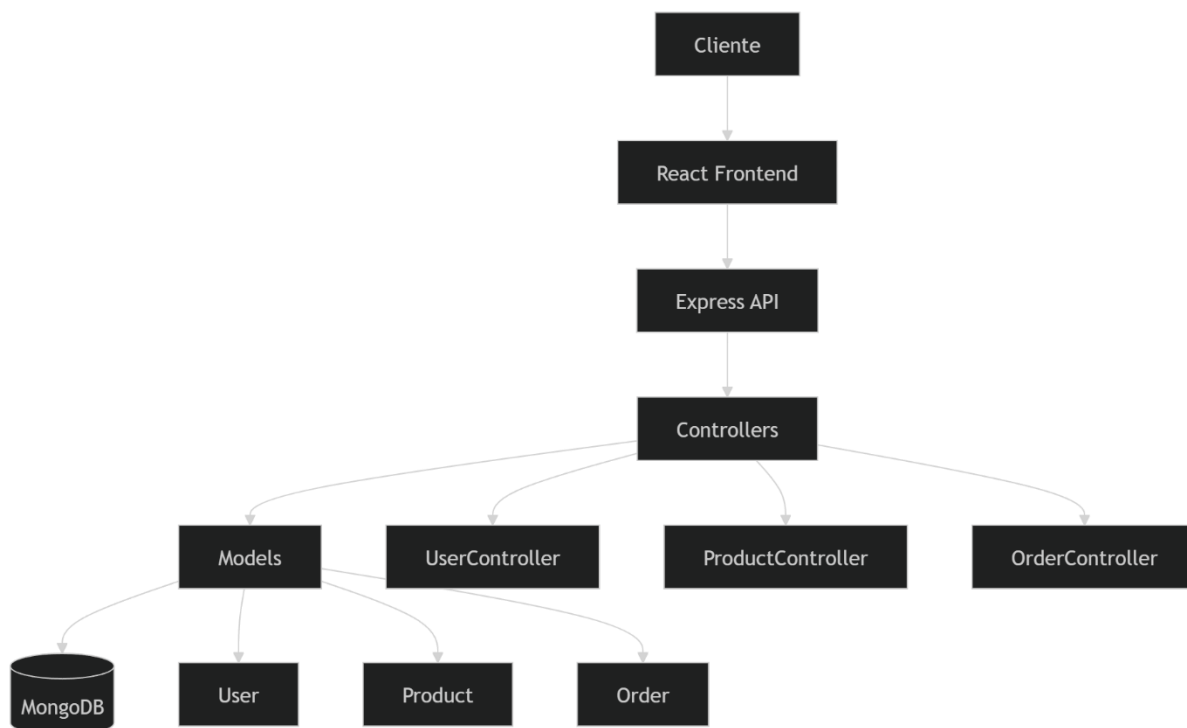


Figura 1 – Diagrama de Arquitetura

Fonte: Autores (2026).

4.2 Diagrama de caso de uso

O diagrama de casos de uso apresenta as principais funcionalidades disponibilizadas pelo sistema e as interações realizadas pelos seus usuários. Por meio dele é possível identificar os recursos oferecidos pela aplicação e os diferentes perfis de acesso existentes.

O sistema possui dois atores principais: Cliente e Administrador. Cada ator possui permissões específicas de acordo com sua função dentro da plataforma.

Cliente

O ator Cliente representa os usuários responsáveis pela navegação e realização de compras no sistema. Entre suas principais funcionalidades estão o cadastro, autenticação, consulta de produtos, gerenciamento do carrinho de compras e acompanhamento de pedidos.



Figura 2 – Diagrama de caso de uso (Cliente)

Fonte: Autores (2026).

Administrador

O ator Administrador possui permissões adicionais para gerenciamento da plataforma. Entre suas responsabilidades estão o cadastro, edição e exclusão de produtos, além do acompanhamento de usuários e pedidos realizados.



Figura 3 – Diagrama de caso de uso (Administrador)

Fonte: Autores (2026).

4.3 Diagrama de Classes

O diagrama de classes representa a estrutura estática do sistema, demonstrando as entidades principais, seus atributos e os relacionamentos existentes entre elas.

A modelagem foi construída com base nas entidades fundamentais da aplicação de comércio eletrônico, responsáveis pelo armazenamento das informações dos usuários, produtos e pedidos.

As principais classes identificadas foram:

User: responsável pelo armazenamento dos dados dos usuários cadastrados.

Product: representa os produtos disponibilizados para venda na plataforma.

Order: armazena as informações referentes aos pedidos realizados pelos clientes.

OrderItem: representa os itens associados a cada pedido.

Os relacionamentos estabelecidos permitem que um usuário realize diversos pedidos, que cada pedido possua vários itens e que um produto possa estar associado a diferentes pedidos.

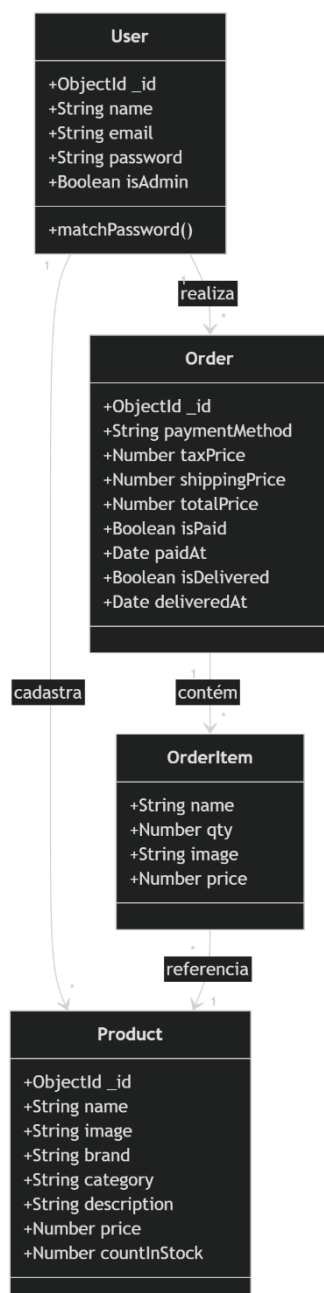


Figura 4 – Diagrama de Classes

Fonte: Autores (2026).

4.4 Diagramas de Sequência

Os diagramas de sequência foram utilizados para representar o fluxo de execução das principais funcionalidades do sistema. Esses diagramas demonstram a troca de mensagens entre os componentes da aplicação e a ordem cronológica das operações realizadas.

4.4.1 Diagrama de Sequência – Login

O processo de autenticação inicia-se quando o usuário informa suas credenciais de acesso. O Frontend envia uma requisição para a API, que valida os dados recebidos junto ao banco de dados. Caso as informações estejam corretas, um token JWT é gerado e retornado ao usuário, permitindo o acesso às funcionalidades protegidas do sistema.

As etapas do processo são apresentadas a seguir:

1. O usuário informa e-mail e senha.
2. O Frontend envia uma requisição POST para a rota */api/users/login*.
3. O UserController recebe a solicitação.
4. O UserModel busca o usuário no banco.
5. O método `matchPassword()` valida a senha.
6. Caso os dados estejam corretos, um token JWT é gerado.
7. O sistema retorna os dados do usuário e o token.
8. O usuário é autenticado.

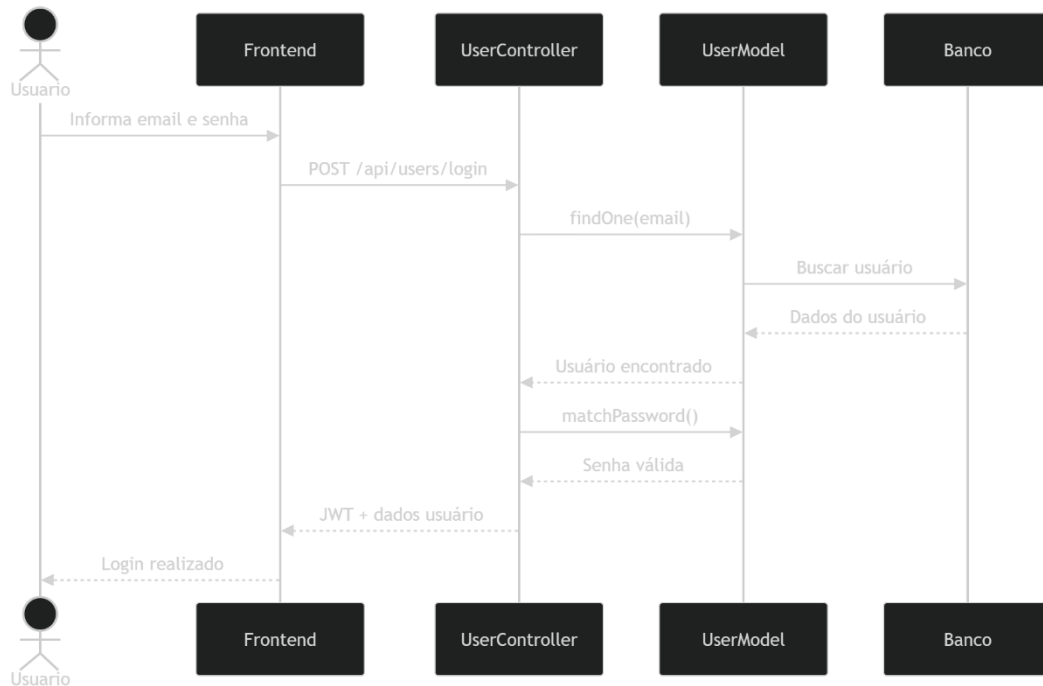


Figura 5 – Diagrama de Sequência (Login)

Fonte: Autores (2026).

4.4.2 Diagrama de Sequência – Compra

O processo de compra ocorre quando o cliente finaliza os produtos adicionados ao carrinho. As informações são enviadas para a API, que cria um novo pedido e realiza seu armazenamento no banco de dados.

As etapas do processo são apresentadas a seguir:

1. O cliente finaliza a compra.
2. O Frontend envia uma requisição para *POST /api/orders*.
3. O OrderController recebe os dados.
4. Um novo pedido é criado.
5. O OrderModel salva as informações no MongoDB.
6. O banco confirma a gravação.
7. O sistema retorna status 201 (Created).
8. A compra é confirmada ao cliente.

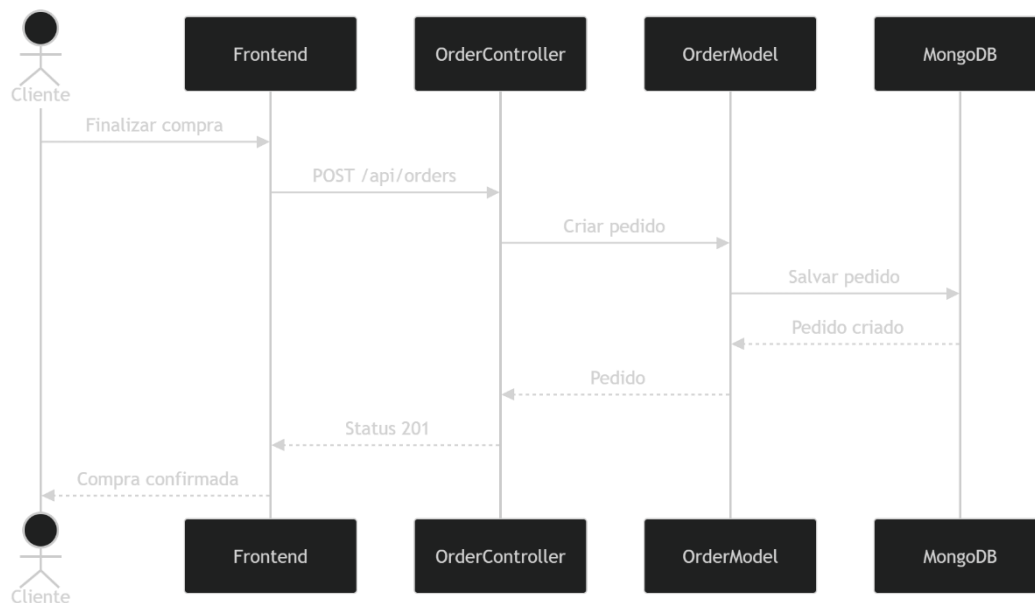


Figura 6 – Diagrama de Sequência (Compra)

Fonte: Autores (2026).

4.5 Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades que o sistema de e-commerce deve oferecer aos seus usuários, definindo os serviços e operações que a aplicação deve executar para atender às necessidades do negócio.

RF01 – O sistema deve permitir o cadastro de usuários.

RF02 – O sistema deve permitir login utilizando e-mail e senha.

RF03 – O sistema deve listar os produtos cadastrados.

RF04 – O sistema deve permitir a pesquisa de produtos.

RF05 – O sistema deve permitir a visualização dos detalhes dos produtos.

RF06 – O sistema deve permitir adicionar produtos ao carrinho de compras.

RF07 – O sistema deve permitir a finalização de pedidos.

RF08 – O sistema deve permitir a consulta do histórico de pedidos realizados.

RF09 – O sistema deve permitir a edição dos dados do perfil do usuário.

RF10 – O sistema deve permitir que administradores realizem o gerenciamento de produtos.

RF11 – O sistema deve permitir que administradores visualizem os usuários cadastrados.

RF12 – O sistema deve permitir que administradores visualizem os pedidos realizados.

4.6 Requisitos Não Funcionais

Os requisitos não funcionais definem as características de qualidade e as restrições técnicas do sistema, estabelecendo critérios relacionados à segurança, desempenho, usabilidade e tecnologia empregada.

RNF01 – O sistema deve ser acessível através dos principais navegadores modernos.

RNF02 – As senhas dos usuários devem ser armazenadas de forma criptografada utilizando BCrypt.

RNF03 – A autenticação dos usuários deve ser realizada por meio de JSON Web Token (JWT).

RNF04 – O sistema deve utilizar o banco de dados MongoDB para armazenamento das informações.

RNF05 – A comunicação entre cliente e servidor deve seguir o padrão de arquitetura REST.

RNF06 – O Frontend deve ser desenvolvido utilizando React.

RNF07– O Backend deve ser desenvolvido utilizando Node.js e Express.js.

5 CONSIDERAÇÕES FINAIS

O desenvolvimento deste projeto permitiu compreender de forma prática os principais conceitos envolvidos na criação de aplicações Full Stack modernas. A integração entre Frontend, Backend e Banco de Dados proporcionou uma visão completa do processo de desenvolvimento de sistemas web.

Por meio da implementação das funcionalidades propostas, foi possível aplicar conhecimentos relacionados à programação, modelagem de sistemas, autenticação de usuários, manipulação de banco de dados e construção de APIs REST.

Os resultados obtidos demonstram que o sistema atende aos requisitos essenciais de uma plataforma de comércio eletrônico, oferecendo recursos para gerenciamento de usuários, produtos e pedidos. Além disso, o projeto contribuiu para o aprimoramento das habilidades técnicas e para a compreensão das boas práticas de desenvolvimento de software.

Como trabalhos futuros, podem ser implementadas funcionalidades adicionais, como integração com meios de pagamento reais, geração de relatórios gerenciais, sistema de avaliações de produtos e melhorias na experiência do usuário.

REFERÊNCIAS

MONGODB. MongoDB Documentation. Disponível em: <https://www.mongodb.com/docs/>.

MONGOOSE. Mongoose Documentation. Disponível em: <https://mongoosejs.com/docs/>.

NODE.JS. Node.js Documentation. Disponível em: <https://nodejs.org/docs/latest/api/>.

EXPRESS. Express.js Documentation. Disponível em: <https://expressjs.com/>.

REACT. React Documentation. Disponível em: <https://react.dev/>.

REDUX. Redux Documentation. Disponível em: <https://redux.js.org/>.

AXIOS. Axios Documentation. Disponível em: <https://axios-http.com/docs/intro>.

JSON WEB TOKEN. JWT Introduction. Disponível em: <https://jwt.io/introduction>.

BCRYPTJS. BCryptJS Documentation. Disponível em: <https://github.com/dcodeIO/bcrypt.js>.

BOOTSTRAP. Bootstrap Documentation. Disponível em: <https://getbootstrap.com/docs/>.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Padrões de projetos: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

OBJECT MANAGEMENT GROUP (OMG). *Unified Modeling Language (UML)*. Disponível em: <https://www.uml.org/>.

IBM. Deployment diagrams. IBM Documentation. Disponível em: <https://www.ibm.com/docs/pt-br/rsas/7.5.0?topic=topologies-deployment-diagrams>.